

LAFC-Evict: A Large-Scale Counterfactual Benchmark for Learned Cache Eviction

Soroush Vahidi^{a,*}

^a*New Jersey Institute of Technology, Newark, NJ, USA*

Abstract

Evaluating cache-eviction decisions requires knowing what would have happened under each candidate choice, information that is not directly observable from a single trace replay. This paper studies LAFC-Evict, a reusable way to construct that counterfactual evidence at production scale – a per-decision, per-candidate finite-horizon counterfactual miss cost, computed for real cache traces – and evaluates it against five questions. Is the target actually discriminative between candidates, and when does it degenerate? Across five trace families, four capacities, and three horizons, we find the target is not uniformly discriminative: two-thirds of decisions are fully tied in aggregate and no audited decision has a strict unique winner, but discriminativeness varies strongly and predictably by workload, capacity, and horizon, and one workload is a fully degenerate, mechanistically explained negative control. Does the offline candidate ranking correspond to actual closed-loop cache performance? A production-scale closed-loop evaluation (LRU, MRU, random, SIEVE; 5 families $\times$ 2 capacities $\times$ 20 seeds) shows offline and closed-loop policy orderings agree in most workload/capacity regimes where the offline signal is not near zero, with two specific, characterized exceptions. How does this depend on workload, capacity, and horizon? Capacity-scale reuse in the underlying trace predicts both offline discriminativeness and closed-loop policy separation ($r = 0.83$ – 0.97 , $n = 10$, exploratory), and a horizon of 16 requests aligns offline and closed-loop evidence more strongly than shorter horizons. How sensitive are the labels to the continuation-policy assumption used to construct them? A

*Corresponding author.

Email address: `sv96@njit.edu` (Soroush Vahidi)

pre-registered, stratified 5,500-decision sampled study at capacities 32 and 128 finds both MRU and mean-random continuation classify robust, and a separate full-population MRU census over capacities 32, 64, 128, and 256 also satisfies the same robustness criterion. Random continuation is measurably more perturbing than MRU in the sample, and no population-scale random census is claimed; we report this with its exact scope rather than as a blanket robustness claim. What does this imply in practice? We argue, and the evidence here supports, that this offline supervision complements rather than replaces closed-loop evaluation, and give concrete guidance for when the offline signal can be trusted. LAFC-Evict also releases the underlying candidate-level, decision-level, and pairwise supervision (277,995,072 candidate rows, 2,363,286 decisions, five trace families) for reuse by other researchers. Every quantitative claim in this paper traces to an independently validated, gate-checked artifact, including the completed and independently rechecked full-population MRU continuation census.

Keywords: learned cache eviction, benchmark datasets, counterfactual supervision, cache replacement

1. Introduction

When a cache request misses and the cache is full, a system must pick one resident object to evict before admitting the requested object. Evaluating whether a given eviction decision was a good one requires knowing what would have happened to each candidate victim if it, rather than another object, had been evicted – information that is not directly observable from a single trace replay, because only one candidate is actually evicted at each decision. This paper studies a reusable way to construct that counterfactual evidence at scale, and evaluates when it is informative, when it degenerates, how it relates to actual closed-loop cache performance, and how sensitive it is to one of its own modeling assumptions.

LAFC-Evict materializes, for real cache traces, a per-decision, per-candidate finite-horizon counterfactual miss cost: for each object that could have been evicted at a full-cache miss, how many additional misses would occur over the next H requests if that object, specifically, had been the one evicted. This candidate-level quantity is stored as `y_loss` (a convenience alias `y_value = -y_loss` is also provided; see Section 8). Section 2 works through one full decision by hand – resident objects, a forced eviction, a future request

window, and the resulting candidate costs, including a case where two candidates tie for optimal – before any aggregate statistic is introduced, so that every quantity used later in the paper is grounded in a concrete example first.

The contribution of this paper is not a new eviction policy. It is an evaluation of a reusable offline supervision surface against closed-loop cache-policy behavior, organized around five questions. **(RQ1)** Is the finite-horizon counterfactual target actually discriminative between candidates, and when does it degenerate to an uninformative tie? We find that it is not uniformly discriminative: aggregate tie rates are substantial, and no audited decision has a strict unique winner, but discriminativeness varies strongly and predictably by workload, cache capacity, and horizon, and one workload in our evaluation is a genuine, mechanistically explained negative control. **(RQ2)** Does the offline candidate ranking correspond to actual closed-loop cache performance? Across a production-scale closed-loop evaluation, offline and closed-loop policy orderings agree in the large majority of workload/ capacity regimes where the offline signal itself is not near zero, with two specific, characterized exceptions. **(RQ3)** How does this correspondence depend on workload, capacity, and horizon? We find that capacity-scale locality in the underlying trace predicts both offline discriminativeness and closed-loop policy separation, and that a horizon of 16 future requests aligns offline and closed-loop evidence more strongly than shorter horizons, though modestly. **(RQ4)** How sensitive are the labels to the choice of continuation policy after a forced eviction? A sampled robustness study at two cache capacities finds the labels robust to MRU and mean-random continuations by a pre-registered criterion; a separate full-population census extends the MRU result across all four evaluated capacities, while still leaving arbitrary continuation policies outside the claim. **(RQ5)** What does this imply in practice for a researcher deciding between offline and closed-loop evaluation? We argue, and the evidence in this paper supports, that the offline supervision this dataset provides complements rather than replaces closed-loop evaluation, and we give concrete, evidenced guidance for when the offline signal can be trusted.

Every quantitative claim in this paper traces to a specific, independently validated evidence artifact; Sections 8 through 9 and the limitations discussion in Section 16 state the exact scope, sample sizes, and caveats attached to each result rather than presenting them as unqualified findings. The underlying release also exposes candidate-level, decision-level, and pairwise derived views of the same supervision at production scale (Table 1), which

supports the evaluation methodology in this paper and is also intended for reuse by other researchers studying learned cache eviction; release construction, schema, and validation are described in Sections 3–7 but are not the paper’s central argument.

To our knowledge, existing public cache-replacement resources do not provide a reusable candidate-level counterfactual supervision surface at this scale together with a direct evaluation of its relationship to closed-loop cache-policy behavior; we make this claim with the restrained scope stated above, not as a claim of exhaustive prior-art coverage (see Section 17).

The rest of the paper proceeds as follows. Section 2 gives the worked example and motivates decision-aligned counterfactual supervision. Sections 3–7 describe construction, label definition, schema, and release validation. Section 8 defines the benchmark tasks. Section 9 reports target discriminativeness (RQ1) and, together with material drawn from the closed-loop production evaluation, addresses RQ2–RQ4 (offline/ closed-loop correspondence, workload/capacity/horizon dependence, and continuation-policy robustness); practical implications (RQ5) are synthesized alongside it. Section 16 states limitations, ethics, and reuse constraints, followed by related work and conclusion.

2. Background and a Worked Example

At each full-cache miss, a caching system must evict one resident object before admitting the requested one. Classical policies such as LRU, ARC [1], and LIRS [2] operationalize this choice through recency/frequency structure; more recent heuristics such as SIEVE [3] and S3-FIFO [4] revisit that structure for scan-resistance and simplicity. A recent hybrid line tunes such heuristics’ own parameters with a learned control plane rather than replacing them outright: LAH, instantiated as S4-FIFO over an S3-FIFO data plane, reports robustness and interpretability gains from this decoupling [5]. Learned or learning-augmented approaches (for example LeCaR [6], LRB [7], Parrot [8], GL-Cache [9], CACHEUS [10], 3L-Cache [11], and HALP [12]) instead attempt to predict which eviction will lead to lower future miss cost, typically by training on labels each system derives for itself from raw traces. This makes cross-paper comparison hard: different papers use different traces, different label constructions, and different evaluation protocols, so a reported improvement is often not attributable to the policy alone.

L AFC-Evict addresses the label-construction half of that problem by materializing a reusable, candidate-level counterfactual label directly from real traces, independent of any specific learned method. The rest of this section works through exactly what that label is, using a small, fully hand-checkable example (full derivation in the paper’s reproducibility artifact); Section 3 then defines the same quantities formally for the production-scale dataset.

2.1. A worked eviction decision

Suppose a cache holds three objects at capacity 3, ordered from least-recently-used to most-recently-used: A, B, C. A request arrives for object D, which is not resident: a *cache miss*. The cache is full, so admitting D requires evicting exactly one resident object. A, B, and C are the three *candidate victims* for this *eviction decision*.

To score each candidate, we look ahead a finite *horizon* of $H = 3$ future requests, which turn out to be B, E, F. For each candidate, we ask: if this candidate, and no other, had been evicted, and the cache had then continued to evict under a *continuation policy* (LRU, the canonical choice throughout this dataset) for the next H requests, how many of those requests would miss? This defines the candidate’s *counterfactual miss cost*, stored as `y_loss` (a convenience alias `y_value = -y_loss` is also provided; see Section 3).

Figure 1 works through all three resulting counterfactual worlds by hand. Evicting A costs 2 future misses (B hits because A’s eviction did not affect B; E and F both miss because they are new objects). Evicting B costs 3 (B itself is now a miss, plus E and F). Evicting C also costs 2, by the same reasoning as A. **The optimal candidate set for this decision is $\{A, C\}$, a tie** – both evictions are equally good by this measure, and only evicting B is strictly worse. This is not a contrived edge case: across the full production dataset, no audited decision has a strict unique winner (Section 9), so tie handling of exactly this kind is the common case, not the exception.

2.2. From one decision to the released views

Grouping every candidate row by its decision id gives the *decision view* (candidate count, optimal-set membership, tie flags, regret summaries). Deriving pairwise preference labels directly from the candidate-level `y_loss` values – e.g. this decision yields (A,B): A preferred, (A,C): tie, (B,C): C preferred – gives the *pairwise derived view*. The pairwise view introduces no new ground truth: every label in it is a function of the candidate view already described. This is a narrower claim than systems such as HALP, which train

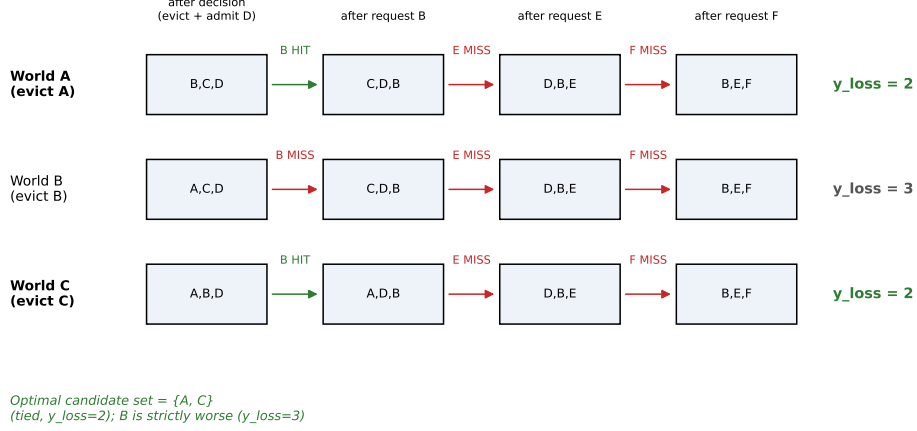


Figure 1: The worked example: three counterfactual worlds (one per candidate victim), each replayed forward under LRU continuation for the next $H = 3$ requests. A and C tie for optimal ($y_loss = 2$); evicting B is strictly worse ($y_loss = 3$).

directly on preference-comparison feedback as their primary learning signal [12]: LAFC-Evict’s pairwise view is instead a deterministic reformatting of the already-defined candidate-level y_loss values, released for compatibility with pairwise/ranking methods rather than as an independent source of supervision (Section 8 discusses why a derived pairwise view is still useful for certain modeling and diagnostic purposes). A single full-cache miss with k admissible candidates therefore appears in three aligned forms: k candidate rows, one decision-group summary, and up to $k(k - 1)/2$ within-decision pairwise comparisons.

3. Evaluation Methodology: LAFC-Evict Construction

This section defines the scientific abstraction underlying LAFC-Evict before describing how it is packaged for release. A *trace* is a sequence of *requests* (accesses) to objects. A *cache state* is the set of objects resident in a bounded-capacity cache at a point in the trace. When a request misses and the cache is full, the system faces an *eviction decision*: choose one resident object, a *candidate victim*, to evict before admitting the requested object. LAFC-Evict treats every such decision, across every trace, capacity, and horizon evaluated, as one supervised unit; Section 8 defines the exact formal quantities

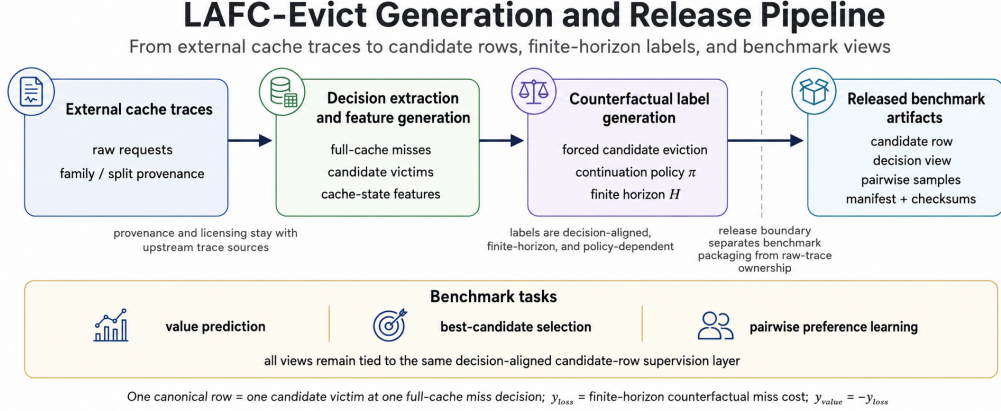


Figure 2: LAFC-Evict generation and release pipeline. External cache traces are converted into decision-level candidate rows, finite-horizon counterfactual labels, and released benchmark views for value prediction, best-candidate selection, and pairwise preference learning.

(finite-horizon counterfactual loss, optimal candidate set, regret) built on top of this abstraction, following directly from the worked example in Section 2.

The cache abstraction used throughout this paper is intentionally narrow and stated explicitly here rather than left implicit: objects are unit-sized (no byte-weighted or variable-size accounting), capacity is counted in objects, and admission of the requested object is unconditional (eviction is never avoided by refusing admission). Section 16 discusses the consequences of this scope boundary; every claim in this paper should be read as bounded by it.

The evaluated dataset covers five trace families – **cloudphysics**, **metacdn**, **metakv**, **twemcache**, and **wiki2018** – chosen to span a CDN workload, a key-value workload, two additional production-derived workloads, and one intentionally included negative/control workload (wiki2018; Sections 9 and 12 show why it degenerates and treat that as a useful, explained result rather than a benchmark defect). Two further families, **brightkite** and **citibike**, are excluded from the open release subset for redistribution reasons; this is a release-packaging boundary, not a claim that the underlying generated corpus is limited to five families. Not every family appears in every split (e.g. metacdn contributes train/validation but no test decisions); split assignment is produced upstream of this release package and is normalized and validated, not assigned, here.

The evaluation in this paper is organized to satisfy four requirements:

1. **Decision alignment.** Preserve within-decision candidate competition

Table 1: Dataset scale from preserved-release metadata. Counts are manifest-backed or view-backed as noted.

metric	value
candidate rows	277,995,072
decision rows	2,363,286
pairwise-sample rows	1,000,000
preserved release size	2.7G
candidate parquet files	168
total release files	176
manifest file_inventory count	176
checksum entry count	175
selected trace families	cloudphysics, metacdn, metakv, twemcache, wiki2018
excluded trace families	brightkite, citibike
full validation status	passed

Table 2: Split counts from manifest-backed candidate-row metadata. These are candidate-row counts, not decision-view counts.

split	row_count
test	28,205,952
train	205,726,368
val	44,062,752

rather than treating candidates as i.i.d. points.

2. **Task multiplicity.** Support pointwise (candidate-level), decision-level, and pairwise views of the same underlying supervision.
3. **Closed-loop grounding.** Evaluate the offline labels against actual closed-loop cache-policy behavior rather than presenting them as self-validating (Sections 10 and 11).
4. **Reproducibility.** Expose validated, checksummed artifacts for every reported number (Section 7).

4. Experimental Methodology

This section consolidates the methodological choices used across every result reported in Sections 9–13 so that scope and caveats are stated once

rather than scattered.

Workloads, capacities, horizons. All results use the five families defined in Section 3 (cloudphysics, metacdn, metakv, twemcache, wiki2018). Offline target-discriminateness results (Section 9) span all four evaluated capacities (32, 64, 128, 256) and all three horizons ($H \in \{4, 8, 16\}$). Closed-loop, linkage, mechanistic, and continuation results (Sections 10–13) are reported at capacities 32 and 128 only – this is the scope of the Tier-1 production closed-loop evaluation on which all three later sections depend, not a data-availability limitation, and it is stated here once rather than repeated in each section.

Split terminology and two family-specific caveats. “test” denotes a temporally held-out scoring window; “validation” denotes a window used during earlier development that is not temporally held out in the same sense. Two caveats apply and recur throughout the paper: (i) **MetaCDN’s closed-loop evidence is validation-window evidence, not test-window**, for both of its evaluated capacities; every result that includes MetaCDN should be read with this in mind, and Section 11 reports what changes if MetaCDN is excluded. (ii) **MetaKV is the only family whose scored window begins at an empty cache** (a cold start, identifiable from the identity *evictions* = *misses* – *capacity* holding exactly for MetaKV and no other family); Section 12 discusses this as a plausible, not established, contributor to MetaKV’s one policy-ranking exception.

Policies and seeds. Closed-loop replay is evaluated for LRU, MRU, SIEVE (all deterministic, one run each), and random (20 CRN seeds, seeds 0–19; mean and per-seed standard deviation reported). SIEVE has no offline counterpart in the labels described in Section 5 and is therefore closed-loop-only evidence throughout – it is not part of any offline-vs-closed-loop correspondence claim, but is still reported as a closed-loop reference point (Section 10).

Offline metrics. Per decision: all-tied fraction, unique-winner fraction, mean/median optimal-set fraction, decision-weighted random-optimal probability, and random regret (mean and tail percentiles), all computed per (family, capacity, horizon) stratum before any pooling (Section 9).

Closed-loop metrics. Miss ratio per (family, capacity, policy), and, for random, its across-seed mean and standard deviation; relative miss-ratio difference vs. LRU is reported as the primary comparison statistic (Section 10).

Offline↔closed-loop linkage metrics. Sign concordance between the offline regret gap and the closed-loop miss-ratio gap (MRU-vs-LRU and

random-vs-LRU only, since these are the two pairs with both an offline and a closed-loop side), plus Pearson/Spearman/Kendall correlation between the two gaps’ magnitudes, computed per horizon on the $n = 10$ (family, capacity) cells (Section 11). $n = 10$ is small and is treated as such throughout: correlations are reported with their exact n , never as population-scale claims.

Continuation-sensitivity metrics. Optimal-set Jaccard overlap, cross-continuation regret (in misses), and strict-reversal fraction, comparing the canonical LRU-continuation label against MRU- and mean-random-continuation (10 CRN seeds) relabelings of the same decisions, on a pre-registered, stratified 5,000-primary + 500-diagnostic decision sample at capacities 32 and 128, plus a full-population MRU census over all five families, capacities 32/64/128/256, and horizons 4/8/16 (Section 13). The primary statistical unit here is the *decision* (not the candidate row or the continuation seed): every reported sampled-study statistic is a decision-level cluster statistic, and the one reported bootstrap confidence interval resamples at the decision level; the MRU census is reported as a complete population enumeration, not an inferential sample.

Tie-aware analysis. Every metric above that could be silently distorted by ties (optimal-set membership, pairwise preference, concordance) is computed with ties as a first-class outcome rather than broken arbitrarily; Section 2’s worked example shows the base case this generalizes from.

Provenance and validation strategy. Every number in this paper traces to a frozen, gate-validated artifact (14/14 gates for Tier-1; independent dual-code-path recomputation for the linkage, mechanistic, and continuation analyses; see Section 7 for the full provenance chain and `docs/PE_CLAIM_EVIDENCE_LEDGER.md` in the accompanying artifact for the exact source of every reported value). No number in this paper is read from a partial or unvalidated intermediate run; the full-population MRU continuation census is used only after its 60/60 chunks, 2,363,286/2,363,286 records, 30/30 validity gates, and independent recheck passed.

5. Counterfactual Label Generation

The benchmark label is defined on candidate victims rather than on completed policies. Let $r_1, r_2, \dots$ denote a request sequence, and let t index a request r_t that misses when the cache is full. Write S_t^- for the cache state immediately before the eviction choice; the candidate set $C_t = S_t^-$ is every resident object, so $|C_t|$ equals the cache capacity (32, 64, 128, or 256 objects

in the evaluated release), counted in objects rather than bytes, with upstream object sizes not separately modeled. For any candidate $c \in C_t$, consider the intervention that evicts c , admits the missed request into the freed slot, and then follows a fixed continuation policy π for the next H requests, resolving every subsequent miss according to π ; in the evaluated release, π is LRU continuation rather than a Belady/offline-optimal rule. The resulting finite-horizon counterfactual loss counts those subsequent misses, so every miss contributes unit cost:

$$L_{t,H}^\pi(c) = \#\{\tau \in \{t+1, \dots, t+H\} : r_\tau \text{ misses in the induced trajectory}\}.$$

The release stores this quantity as `y_loss`, and it stores the corresponding value label as `y_value` = $-\text{y_loss}$. The triggering miss at time t is common to all candidates and is therefore not the decision-discriminating part of the label.

Both columns are retained in the schema, but they are not independent information: `y_value` is a deterministic sign-flip of `y_loss`, computed by the same generation step. The two exist because downstream modeling code is written against either a minimization convention (`y_loss` as a cost/regret target, the convention used throughout the rest of this paper and the more directly interpretable of the two – a count of extra misses) or a maximization convention (`y_value` as a value/utility target, convenient for argmax-based candidate selection or value-function-style formulations). This paper treats `y_loss` as canonical and uses `y_value` only where a specific baseline is more naturally stated as a maximization, never reporting the same statistic under both names.

This label has two consequences for the paper. First, it creates a common numerical target for regression-style benchmark tasks. Second, it induces structured decision-level objects. For a fixed decision (t, S_t^-, C_t) , the benchmark-optimal candidate set is

$$C_t^\star = \arg \min_{c \in C_t} L_{t,H}^\pi(c),$$

and the per-candidate regret is

$$\Delta_{t,H}^\pi(c) = L_{t,H}^\pi(c) - \min_{c' \in C_t} L_{t,H}^\pi(c').$$

These quantities define best-candidate selection targets, pairwise preferences, and tie structure within a decision.

This supervision is a generated counterfactual label family, not an upstream annotation. It is finite-horizon, not asymptotic. It is tied to a continuation policy, not a universally optimal control objective. The label is therefore decision-aligned for the benchmarked eviction choice, but it is not an offline-optimal label in the sense of solving the full future control problem. These boundaries are central to the scientific positioning of the paper.

6. Dataset Schema and Views

The canonical candidate schema currently consists of decision metadata, candidate identity, split information, labels, and feature columns. Each row corresponds to exactly one candidate victim at exactly one full-cache miss decision. Trace identity and split metadata are recorded through `trace_name`, `trace_family`, `dataset_source`, and `split`. Cache configuration is recorded through `capacity` and `horizon`. Decision identifiers include `decision_id` and `decision_t`. The schema also records a per-decision chunk ID. That field is named `decision_chunk_id`. Candidate identity is represented through the field `candidate_page_id`. The labels are `y_loss`, the finite-horizon counterfactual miss cost after forcing that candidate eviction under the fixed continuation policy, and `y_value = -y_loss`. Feature columns cover request context, candidate context, predictor/LRU signals, cache summary statistics, and recent request or hit rates.

Beyond candidate rows, the release defines two derived views. The *decision view* aggregates all rows that share a decision identifier and summarizes each decision with candidate count, minimum loss, maximum value, the set of optimal candidate IDs, tie count, and regret summary statistics. The *pairwise view* compares candidate pairs from the same decision and derives pairwise preference labels from the ordering of their `y_loss` values, with ties when the losses are equal. The evaluated open release does not ship the full pairwise view; it ships a capped pairwise sample of 1,000,000 rows instead.

Operationally, the schema can be read as a layered projection over the same decision events: candidate rows are the atomic supervised objects, the decision view groups those rows by `decision_id`, and the shipped pairwise sample keeps only selected within-decision comparisons. Table 3 summarizes those column groups and derived fields directly in manuscript form.

Table 3: Compact summary of candidate-row schema groups and derived-view fields.

Column group	Notes
Decision metadata	family, capacity, horizon, split, IDs
Candidate identity	candidate page ID
Labels	<code>y_loss</code> , <code>y_value</code>
Feature groups	request, candidate, cache, disagreement, rate
Derived decision fields	tie and regret summaries
Derived pairwise fields	A/B labels and regret differences

7. Release Construction and Validation

The release pipeline is designed to package existing generated candidate rows into a benchmark artifact without requiring raw traces in the release package. The builder uses an out-of-core DuckDB path to ingest candidate shards, normalize split values, emit partitioned Parquet candidate rows, derive the decision view, optionally create a capped pairwise sample, and write release metadata and checksums.

The validation surface is part of the benchmark contribution. The validation workflow includes:

- schema completeness for candidate partitions,
- split normalization and split-set consistency,
- presence of selected families and absence of blocked families,
- duplicate candidate detection within decision groups,
- consistency of decision metadata within a decision,
- decision-view schema and count consistency,
- checksum verification.

The evaluated open-release snapshot passed metadata repair, the provided verification suite, publication-bundle validation, and end-to-end release validation. The validation evidence comes from manifest-backed counts, schema checks, split checks, duplicate detection, and checksum verification implemented in reproducible scripts. The manuscript therefore reports validation of the release package without relying on unpublished hosting details.

Table 4: Release-package layout using release-relative paths for the evaluated benchmark snapshot.

artifact	relative_path
candidate rows	data/candidate_rows/
decision view	data/decision_view/decision_view.parquet
pairwise sample	data/pairwise_sample/pairwise_sample.parquet
release manifest	metadata/release_manifest.json
checksums	metadata/checksums.sha256

It does not claim that final public hosting or DOI assignment is complete. It also does not treat superseded status notes as the authoritative source of scientific evidence.

Table 4 summarizes the release-package components used in the evaluation: candidate rows, derived benchmark views, manifest metadata, and checksums. Validation claims rely on those package contents together with reproducible validation scripts rather than on any standalone status note.

7.1. Provenance of the evaluation results in this paper

Beyond the released dataset artifact above, every result reported in Sections 9–13 traces to an independently frozen, gate-validated analysis artifact, each with its own committed scripts, provenance record, and (where applicable) an independent second code path that recomputes headline numbers and requires exact agreement: the target-discriminateness audit, the Tier-1 closed-loop production evidence (14/14 validity gates; a byte-verified evidence freeze recording the SHA256 of every output file both before and after freezing), the offline↔closed-loop linkage analysis, the mechanistic workload analysis, and the continuation-policy sensitivity pilot and full sampled study (prelaunch LRU-equivalence gate; post-completion validity gates; an independent recheck script requiring 10^{-9} numerical agreement with the primary analysis), plus the full-population MRU continuation census (60/60 chunks, 2,363,286 decision-horizon records, 30/30 validity gates, independent recheck passed). All six are scripted, reproducible, and checked into the repository alongside this manuscript as compact evidence artifacts.

Two label sources must not be conflated. The candidate-level `y_loss/y_value` labels released and evaluated throughout this paper use the canonical LRU continuation policy (Section 5). The continuation-sensitivity study (Section 13) additionally computes MRU- and random-continuation

relabelings of sampled decisions and an MRU-continuation relabeling of the full census population, strictly for the purpose of measuring sensitivity – these sensitivity-study outputs are not an alternative release and are not used anywhere in this paper as if they were the canonical label.

The population-scale MRU continuation census is frozen evidence. The census run 20260914T042528Z_1a29e773a113 is complete, validated, independently rechecked, and frozen via `analysis/continuation_policy_mru_population_census_20260914/validated/20260914T042528Z_1a29e773a113/EVIDENCE_MANIFEST.json`. The raw 1.6 GB census output is preserved separately outside this manuscript branch; the manuscript imports only compact summaries, validation reports, manifests, and Figure 6 source data.

A known, unresolved provenance limitation, carried forward honestly rather than hidden: the exact historical commit provenance of the full-release label generator is incomplete. This does not affect the validity of the frozen analysis artifacts above, each of which independently records its own generation provenance, but it means the full historical lineage of the generator code itself predating those artifacts is not completely reconstructable from repository history alone.

8. Benchmark Tasks and Baselines

L AFC-Evict supports four benchmark tasks from one release.

Candidate-level value prediction.. Predict `y_loss` or `y_value` from a single candidate row. This is a pointwise task over the canonical benchmark unit and measures how well a model predicts finite-horizon counterfactual eviction quality for one candidate at one decision.

Best-candidate selection.. Given all candidate rows for one decision, identify a member of the minimum-loss set. This task can be framed as ranking or decision-level classification and connects most directly to the actual cache-eviction choice because it evaluates whether a method selects a benchmark-optimal victim within that decision.

Pairwise preference prediction.. Predict whether candidate A is better than candidate B within the same decision, where “better” means lower `y_loss` or equivalently higher `y_value`. This task uses the pairwise view or the shipped pairwise sample and provides a simpler learning interface for within-decision

preference modeling. Because equal losses induce ties, reports using this task need to state explicitly whether evaluation includes tie rows, excludes them, or maps them to a separate class. Within the shipped pairwise sample, the A/B assignment is randomized by a deterministic hash over stable decision and pair-identity fields (`orientation_method: deterministic_hash_v1`), not by candidate-identifier ordering, so the non-tie `label_a_better`/`label_b_better` split is balanced (about 49.8%/50.2%) rather than reflecting a construction-induced directional imbalance.

Candidate-label distribution characterization.. Summarize the empirical label surface without training a predictive model. This characterization task studies distributions of `y_loss`, `y_value`, regret, tie frequency, and decision ambiguity across families, capacities, horizons, and splits. It is descriptive rather than competitive, but it is important for understanding benchmark difficulty and for interpreting downstream task results.

The empirical scope of this paper remains intentionally narrow. Rather than presenting a large model bake-off, we report lightweight baselines that can be executed directly from the committed release views and that establish reproducible reference points for later work.

The current results surface covers three such references. A linear regression baseline over candidate rows predicts `y_loss` from released features. It is intentionally weak: test MAE is 3.8739 with RMSE 4.7152 and $R^2 = 0.0026$, and the validation-split R^2 is slightly negative (-0.0506). Using that same linear score for within-decision selection nevertheless provides a reproducible decision-level floor. On the pairwise task, the earlier random and majority non-tie results remain useful sanity checks, but by themselves they do not test whether the released candidate features contain decision-relevant signal.

To probe that question without changing the released benchmark schema, we build an augmented feature view for evaluation only by joining released candidate-side features back onto the capped shipped pairwise sample. This produces a pairwise-sample-backed evaluation surface rather than a full-pairwise-universe-backed one, and no full quadratic pairwise materialization is performed. The shipped sample used throughout this paper is the canonical regenerated pairwise sample (SHA256 prefix `1d770c69...9eda02`, full digest and provenance in the accompanying reproducibility artifact), not the historical, noncanonical sample that a stale six-column decision-selection key produced in an earlier release artifact; the regenerated sample’s tie-count balance below (near-50/50 non-tie split) is itself evidence of this, since the his-

torical sample’s non-tie rows were heavily skewed (about 6,134 vs. 113,898) in a way traceable to the key bug, not to the underlying label distribution. It is itself tie-heavy: 878,262 of its 1,000,000 rows (87.8%) are exact ties, leaving 121,738 non-tie rows evaluated below. Ties remain common and are reported here as part of the label-surface characterization rather than discarded. Table 6 reports six non-tie pairwise baselines on that 121,738-row surface: the random and majority sanity checks, 1-D LRU-score and predictor-score models, a pairwise baseline induced by the released point-wise linear `y_loss` regressor, and a logistic regression over released feature differences.

The resulting evidence is stronger than the earlier sanity checks alone. The logistic feature-difference model is the strongest pairwise baseline, reaching 0.9660 accuracy with 0.0824 log loss on the non-tie subset of the shipped sample. The linear-score and LRU-score baselines also outperform the random and majority checks, which indicates that the released candidate features retain decision-relevant signal for pairwise preference prediction. These numbers nevertheless warrant conservative interpretation. They improve the empirical benchmark evidence without changing the released candidate-row, decision-view, or shipped pairwise artifacts, and they do not support full-universe or online-policy claims.

The best-candidate evaluator groups candidate rows by the canonical 9-column decision key and reports 2,363,286 decisions, an exact match with the decision view (Section 3). A cheap decision-level floor computed directly from the decision view shows why this match matters for interpretation: a uniform-random selector’s expected optimal-selection rate is 0.9912 with expected regret 0.0088, both more favorable than the linear-score selector’s observed 0.8753 optimal-selection rate and 0.1251 mean regret, purely because of the tie density documented in Section 9 – not because the linear score underperforms a stronger baseline. Headline selection-rate and regret numbers should therefore always be read alongside this random floor rather than in isolation.

9. Target Discriminativeness (RQ1)

This section answers RQ1: is the finite-horizon counterfactual target discriminative between candidates, and when does it degenerate? Source: `analysis/sigmoid_target_discriminativeness_20260913/`, audited over

Table 5: Pairwise-sample-backed summary statistics for the shipped sample view.

statistic	value
pairwise rows	1,000,000
split breakdown	test=102,024; train=737,896; val=160,080
family breakdown	wiki2018=250,872; cloudphysics=245,336; metakv=190,584; twemcache=169,064; metacdn=144,144
unique decisions represented	118,635
label distribution	a_better=60,673; b_better=61,065; tie=878,262
A/B orientation method	deterministic_hash_v1 (randomized, not lexicographic)

all 277,995,072 candidate rows / 2,363,286 decisions in the canonical dataset (all capacities, all horizons; the full population, not a sample).

9.1. The uncomfortable aggregate facts, stated first

Pooled across every family, capacity, and horizon: the decision-weighted random-optimal probability is 0.9912, the decision-weighted mean random regret is 0.0088, the all-tied fraction is 0.6766, and the unique-winner fraction is exactly 0.0. In plain terms: a uniformly random candidate choice lands in the optimal set 99.1% of the time, over two-thirds of decisions have every candidate exactly tied, and *no audited decision in this dataset has a single, strictly-best candidate victim*. The overall classification, by the audit’s own pre-registered rule, is **B = CONDITIONALLY DISCRIMINATIVE** – not uniformly discriminative, and this is stated here as the paper’s own finding rather than a limitation discovered post hoc. Aggregate degeneracy this large is, by itself, a substantive result about the target’s shape.

9.2. Where the target is informative

The aggregate picture, however, is not the same at every family, capacity, and horizon – consistent with prior large-scale production evidence that cache workloads vary substantially in access pattern and reuse structure even within a single deployment [13], which is why this section reports per-family results rather than only the pooled figure above. Figure 3 shows discriminativeness (1 – all-tied fraction) across all five families, four capacities, and three horizons (source data: same audit, per-stratum breakdown, `outputs/phase4_stratified_family_capacity_horizon.csv`, 60 family×capacity×horizon

Table 6: Feature-based pairwise baselines on the non-tie subset (121,738 of 1,000,000 rows) of the shipped pairwise sample, evaluated after the A/B orientation fix (deterministic hash-based randomization, not lexicographic ordering). An evaluation-only augmented feature view joins released candidate features back to the capped sample. Results are pairwise-sample-backed rather than full-pairwise-universe-backed, and no full quadratic pairwise materialization is performed.

baseline	feature source / model type	accuracy	log loss
random non-tie	random-label sanity check	0.5006	0.6931
majority non-tie	empirical majority-class sanity check	0.5016	0.6931
LRU-score pairwise	1-D logistic on LRU-score difference	0.9225	0.2426
predictor-score pairwise	1-D logistic on predictor-score difference	0.5016	0.6931
linear-score pairwise	logistic on pointwise linear-score difference	0.9490	0.1161
logistic feature difference	logistic regression on released feature differences	0.9660	0.0824

cells). Two families, **metacdn** and **twemcache**, are substantially more discriminative than the aggregate figure suggests – at $H = 16$, metacdn’s all-tied fraction is 0.0085–0.012 and twemcache’s is 0.10–0.24 across its two smallest capacities, an order of magnitude below the aggregate 0.68. **cloudphysics** and **metakv** sit at the tie-heavy end (all-tied fraction 0.66–0.87 at $H = 16$), closer to but still below the degenerate extreme. **wiki2018** is exactly at the degenerate extreme: all-tied fraction 1.0 at every capacity and every horizon tested, with zero reuse events in its scored windows (Section 12 shows this is a mathematical certainty, not a statistical property of the trace, and treats wiki2018 as a genuine negative control rather than a benchmark failure). Discriminativeness also increases with horizon within every non-degenerate family (e.g. twemcache/cap32 rises from 0.46 at $H = 4$ to 0.75 at $H = 16$) – a longer look-ahead window resolves more ties, consistent with more future requests giving more opportunities for candidates’ fates to diverge.

9.3. Interpretation

The scientific contribution here is partly *characterizing* where the target is informative rather than asserting it is informative everywhere. High tie rates are not hidden or explained away: they are the majority regime in this dataset, and Section 12 connects them directly to trace-level reuse struc-

Table 7: Benchmark task definitions and representative evaluation outputs.

Task	Inputs	Representative outputs
Value regression	candidate rows	MAE, RMSE, and optional rank correlation between predicted and observed values
Best-candidate selection	decision groups	Top-1 accuracy against the benchmark-optimal set and mean realized regret relative to the best available candidate
Pairwise preference	pairwise rows from one decision	Accuracy or ROC-AUC, with tie handling stated explicitly and non-tie-only reporting marked when used
Label characterization	candidate or decision aggregates	Histograms, quantiles, tie rates, regret summaries, and per-family/capacity/horizon breakdowns

ture (capacity-scale locality), giving a mechanistic account of exactly why metacdn/twemcache are discriminative and cloudphysics/metakv/wiki2018 are not. Sections 10 and 11 then ask whether this same discriminativeness structure predicts anything about actual closed-loop cache-policy behavior (it does, in the regimes identified here).

10. Closed-Loop Policy Evaluation (RQ2, part 1)

Section 9 characterizes the offline target. This section reports the actual closed-loop cache-policy behavior it will be compared against in Section 11. Source: Tier-1 production closed-loop evaluation (repository path `analysis/closed_loop_tier1_evidence_20260914/`; RUN_ID `20260914T023445Z_8a4cd32402a1`; 14/14 validity gates passed; 230/230 planned executions complete, 0 failed). Scope: 5 families $\times$ 2 capacities $(32, 128) \times 4$ policies (LRU, MRU, random-mean over 20 seeds, SIEVE [3]), per Section 4.

LRU is the best policy in 9 of 10 cells. The sole exception is metakv/cap128, where SIEVE (miss ratio 0.6982) beats LRU (0.7290) by 4.22% relative – the only cell in the entire evaluation where a non-LRU policy wins outright. **MRU is the worst policy in every non-degenerate cell**, by a wide margin (relative miss-ratio increase over LRU ranging from 1.65% at cloudphysics/cap32, a near-zero-signal regime, up to 54.93% at metacdn/cap128). **Random-seed variability is small relative to policy gaps** throughout: the largest random-policy standard deviation observed is

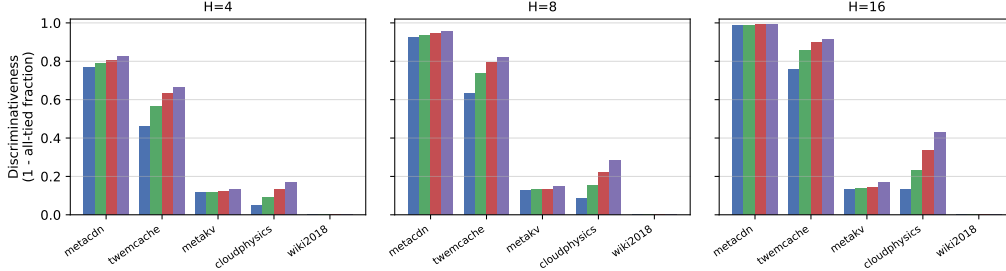


Figure 3: Target discriminativeness (1 - all-tied fraction) by family, capacity, and horizon. Reproducible from `paper/performance_evaluation/scripts/figures/make_figure2_discriminativness.py` against the frozen audit CSV; no number is hand-entered.

0.0025 (twemcache/cap32), an order of magnitude below the smallest genuine policy gap reported in the table.

wiki2018 is a complete, exact tie across every policy: miss ratio 1.0 for LRU, MRU, SIEVE, and every one of random’s 20 seeds, at both capacities, with random’s standard deviation exactly 0.0. This is not approximate degeneracy – it is byte-for-byte identical across all 4 policies and 20 seeds. Section 12 shows this follows deductively from zero reuse events in the scored window, not as a statistical artifact of this particular closed-loop run.

cloudphysics is a near-tie regime at both capacities: every policy’s relative gap to LRU stays under 4.5%, an order of magnitude smaller than metacdn’s or twemcache’s gaps. **The metakv/cap128 cell is the one genuine exception to “LRU dominates”,** and there both SIEVE (miss ratio 0.6982) and random-mean (approximately 0.7137, relative gap -2.07%) beat LRU (0.7290), while MRU remains worst; Section 12 investigates, but does not fully resolve, why. metacdn’s results should be read as validation-window evidence throughout (Section 4); twemcache’s are test-window.

No universal middle-of-the-pack ranking is claimed for SIEVE vs. random: SIEVE beats random-mean at metacdn and metakv/cap32, but random-mean beats SIEVE at twemcache (both capacities), cloudphysics (both capacities), and metakv/cap128 – the relative order of these two policies is family-dependent, not fixed. Only one ordering statement holds across every non-degenerate cell: **MRU is worst everywhere.** LRU is best in 9 of 10

Table 8: Tier-1 closed-loop production evaluation: LRU miss ratio and relative miss-ratio difference vs. LRU for MRU, mean-random (20 seeds), and SIEVE. Source: [analysis/closed_loop_tier1_evidence_20260914/](#) (230/230 complete, 14/14 gates passed). MetaCDN is validation-window evidence; all other families are test-window. Negative values mean the policy beats LRU.

family	cap.	LRU miss ratio	MRU Δ	random Δ	SIEVE Δ	random std
metacdn	32	0.5792	+47.37%	+5.38%	+0.81%	0.0013
metacdn	128	0.5490	+54.93%	+2.98%	+0.36%	0.0014
twemcache	32	0.7511	+27.12%	+4.19%	+8.47%	0.0025
twemcache	128	0.6205	+52.98%	+7.78%	+15.29%	0.0023
metakv	32	0.7410	+10.02%	+0.47%	+0.03%	0.0011
metakv	128	0.7290	+8.57%	-2.07%	-4.22%	0.0015
cloudphysics	32	0.9836	+1.65%	+0.28%	+0.20%	0.0010
cloudphysics	128	0.9561	+4.48%	+0.85%	+1.44%	0.0014
wiki2018	32	1.0000	0.00%	0.00%	0.00%	0.0000
wiki2018	128	1.0000	0.00%	0.00%	0.00%	0.0000

cells and falls only as low as third place, at metakv/cap128, in the tenth.

11. Offline $\leftrightarrow$ Closed-Loop Correspondence (RQ2, part 2)

This section answers the second half of RQ2 – does the offline candidate ranking from Section 9 correspond to the closed-loop behavior in Section 10? – and, jointly with the horizon comparison below, RQ3’s horizon question. Source: [analysis/closed_loop_offline_linkage_20260914/](#), computed with two independent code paths in numerical agreement. **SIEVE has no offline counterpart** (Section 5 defines labels only for the continuation-policy family LRU/MRU/random) and is therefore excluded from every comparison in this section; only MRU-vs-LRU and random-vs-LRU are compared.

RQ-CL1 (rank agreement). At $H = 16$, across the 10 family $\times$ capacity cells: MRU-vs-LRU agrees in sign (offline regret-gap sign matches closed-loop miss-ratio-gap sign) in 7 of 10 cells, disagrees in 1, and is tied on both sides (wiki2018) in 2; random-vs-LRU agrees in 6, disagrees in 2, ties in 2. The discordant cells are always the same two: **cloudphysics/cap32** (discordant for both pairs – offline regret gaps there are -0.0027 and -0.0007 , i.e. barely non-zero on the offline side too) and **metakv/cap128** (discordant for random-vs-LRU only – offline says random is very slightly worse than LRU, $+0.0013$ regret gap, while closed-loop shows random modestly beating LRU,

−0.015 miss-ratio gap; Section 10 additionally notes SIEVE also beats LRU there). Excluding wiki2018’s two trivially-tied cells, concordance among the 8 non-degenerate cells is not manufactured by wiki2018’s automatic agreement: dropping wiki2018 still leaves the same two discordant cells and the same qualitative picture.

RQ-CL2 (gap-magnitude correspondence). At $H = 16$: MRU-vs-LRU Pearson $r = 0.835$ (Spearman $\rho = 0.915$, Kendall $\tau = 0.773$); random-vs-LRU $r = 0.755$ ($\rho = 0.829$, $\tau = 0.591$). Figure 4 shows both relationships directly, with the two discordant cells annotated rather than hidden. These correlations rest on only 8–10 independent (family, capacity) points and are reported with that n attached everywhere; they are not treated as population-scale estimates.

RQ-CL3 (workload/capacity dependence). metacdn and twemcache are the strong-agreement, most-discriminative-offline regimes and also show the largest closed-loop policy separation; cloudphysics is a weak/ambiguous regime where the offline gap is close enough to zero that its sign is not robust; metakv/cap128 is the one reversal regime; wiki2018 is degenerate on both sides, exactly, confirming it as a clean negative control. An exploratory (not pre-registered) correlation between offline discriminativeness ($1 - \text{all-tied fraction}$) and closed-loop $|\text{MRU} - \text{LRU}|$ separation, across all 10 cells at $H = 16$, gives Pearson $r = 0.968$ – Section 12 traces this to a shared underlying trace-locality property rather than treating it as a coincidence.

RQ-CL4 (horizon). $H = 16$ wins or ties on every magnitude-sensitive statistic across every sensitivity slice tested (primary, excluding wiki2018, test-window-only), for both pairs. The coarse concordant/ discordant/tied counts are *identical* at $H = 4$, $H = 8$, and $H = 16$ (the same two cells are discordant regardless of horizon); horizon changes the *strength* of the correspondence (the correlation coefficients above), not which cells agree in sign. The differences between horizons are modest in absolute terms ($r = 0.746$ at $H = 4$ to $r = 0.835$ at $H = 16$ for MRU-vs-LRU) and $n = 8$ –10 per horizon is small, but the pattern is consistent and not cherry-picked.

Answer to RQ2/RQ3. Offline and closed-loop policy orderings agree in the large majority of workload/capacity regimes where the offline signal itself is not near zero, with two specific, characterized exceptions; $H = 16$ gives the strongest, though modest, alignment among the three horizons tested. This is a bounded, evidenced claim, not a universal predictive-validity claim for the offline labels.

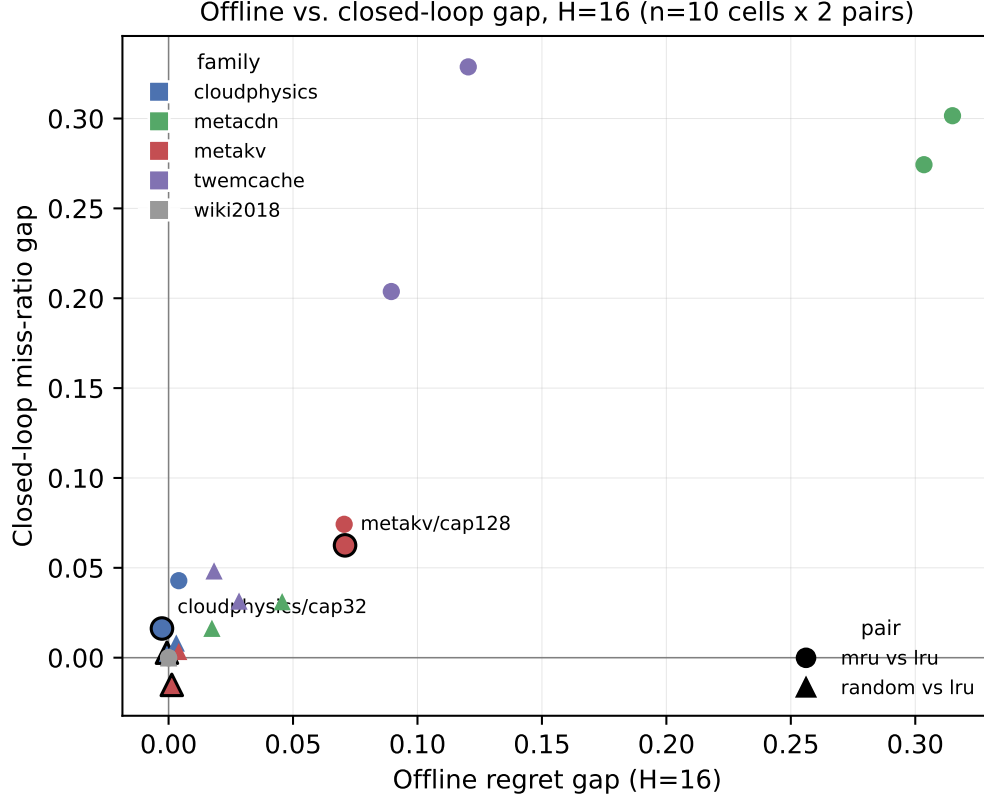


Figure 4: Offline regret gap vs. closed-loop miss-ratio gap, $H=16$, MRU-vs-LRU and random-vs-LRU. Reproducible from `paper/performance_evaluation/scripts/figures/make_figure3_linkage_scatter.py` against `outputs/rq_cl2_scatter_data.csv`.

12. Mechanistic Workload Analysis (RQ3, part 2)

Sections 9 and 11 show *that* discriminativeness and offline $\leftrightarrow$ closed-loop agreement vary by workload; this section asks *why*, using only existing trace-only and closed-loop artifacts (no new simulation). Source: repository path `analysis/closed_loop_mechanistic_analysis_20260914/`. Classifications follow the source report's own discipline exactly: DIRECTLY OBSERVED (a deductive or exactly-reproduced fact), CONSISTENT WITH MECHANISM (plausible, structurally supported, not proven), and CAUSALLY ESTABLISHED (reserved for one deductive result below). No claim here is upgraded beyond its

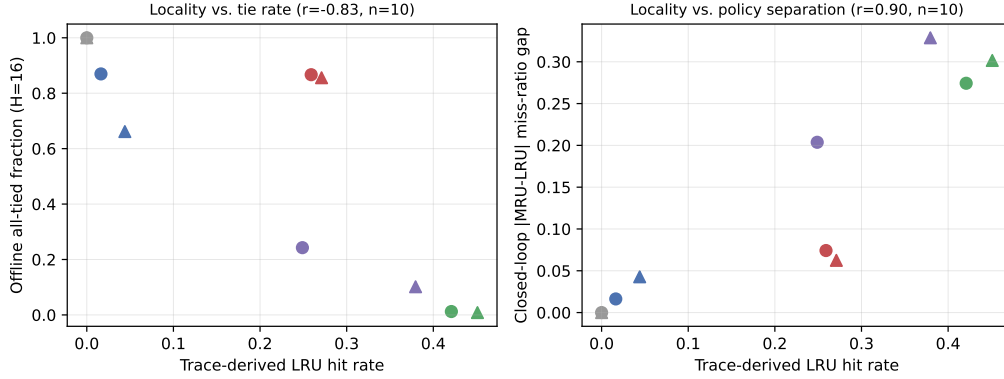


Figure 5: Trace-derived LRU hit rate vs. offline discriminativeness (left) and closed-loop policy separation (right), $n = 10$ family $\times$ capacity cells. Reproducible from `paper/performance_evaluation/scripts/figures/make_figure4_locality.py` against `outputs/mechanism_comparison.csv`.

source classification.

Capacity-scale reuse predicts both discriminativeness and closed-loop separation. Trace-derived LRU hit rate (an exact, capacity-scale reuse measure, computed independently of the labels) correlates with offline discriminativeness at $r = -0.83$ and with closed-loop $|\text{MRU} - \text{LRU}|$ separation at $r = 0.90$ (pooled $n = 10$). Figure 5 shows both relationships directly. This is an $n = 10$, exploratory finding, consistent with but not independent evidence from RQ-CL3’s own exploratory correlation (Section 11) – the same underlying trace property is driving both.

cloudphysics/cap32 (directly observed): a near-zero-signal regime, not a real contradiction. Only 17.5% of window requests are reuses at all; of those, only 9.4% fall within stack distance 32. The trace-only predicted LRU hit rate ($0.175 \times 0.094 = 1.6\%$) exactly reproduces the observed Tier-1 LRU hit rate ($134/8192 = 1.64\%$). The offline regret gaps there (-0.0027 , -0.0007) are tiny on a scale where metacdn’s own MRU-LRU gap is over $100\times$ larger. The Section 11 discordance at this cell is exactly what a coin-flip-scale signal would produce.

metakv/cap128 (consistent with mechanism, not established): a real but bounded exception, characterized, not explained. MetaKV’s reuse-distance median is short (2, comparable to metacdn’s 3) but its tail

is markedly heavier: reuse-within-32 is only 0.574, rising to just 0.600 by distance 128 – a narrow, 2.6-percentage-point “boundary zone” sitting almost exactly at the tested capacities, where a recency-strict policy (LRU) and a second-chance policy (SIEVE) can plausibly diverge. MetaKV’s cold-start scoring window (Section 4) is a second, independently real structural property of this same cell. **Neither is shown to be the cause:** no per-request SIEVE or random trajectory log exists in the frozen artifacts to isolate which property (or their interaction, or neither) produced the reversal, and the source analysis states this explicitly rather than forcing a resolution.

Twemcache’s random-over-SIEVE ordering is not explained by existing data. Twemcache has the strongest “small hot set plus long tail” bimodal signature in the dataset (longest novel streak 3; reuse distance p10=3 but p75=7379), which would naively predict SIEVE’s second-chance admission should help – but the observed result is the opposite (SIEVE loses to random at both capacities). Existing trace-only data can describe this structure but cannot explain the direction of the outcome; the source analysis classifies any “SIEVE benefits from bimodality” story here as NOT SUPPORTED by the data, and this paper does not repeat that story.

wiki2018 (causally established): the one exception to the paper’s general classification discipline, and deliberately so. Zero reuse events occur in the scored window ($n_{\text{reuse}} = 0$ across all 4096 requests, every one a first-ever occurrence over the full 50,000-request trace). Since a hit requires a repeated reference to an already-resident item, miss ratio = 1.0 for every policy is a mathematical certainty given zero reuse events, not a statistical tendency – an existence proof from a directly counted quantity, which is why this one result is classified CAUSALLY ESTABLISHED while every other mechanism in this section is not.

13. Continuation-Policy Sensitivity (RQ4)

The canonical labels (Section 5) use LRU as the continuation policy after a forced eviction. This choice matters because a deployed learned policy would control later evictions, not necessarily LRU. RQ4 therefore asks whether the optimal-candidate set and regret ordering induced by the canonical labels are materially changed when the same forced eviction is followed by a different continuation policy.

Sampled MRU/random study. The pre-registered sampled study (analysis/continuation_policy_sensitivity_full_20260914/, RUN_ID

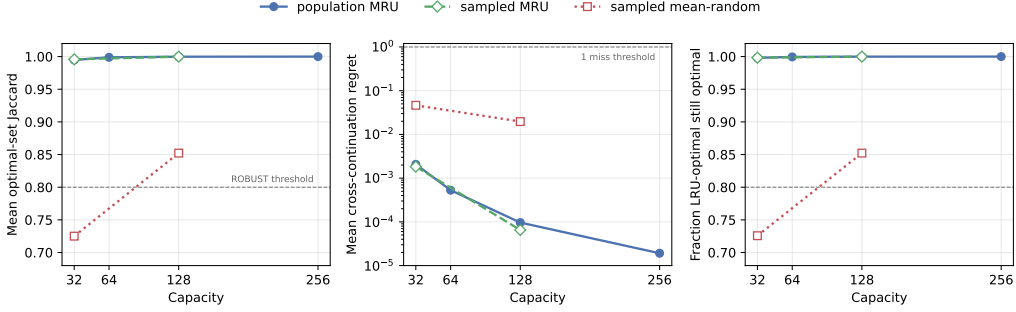


Figure 6: Continuation-policy sensitivity in the pre-registered informative stratum (Set C: decisions discriminative under LRU). Sampled MRU and mean-random evidence covers capacities 32 and 128 only; full-population MRU evidence covers capacities 32, 64, 128, and 256. No population-scale random-continuation result is shown or implied.

20260914T040333Z_1f66342be435) evaluates 5,000 primary decisions plus 500 diagnostic oversample decisions at capacities 32 and 128. Its prelaunch LRU-equivalence gate audited 1,271,616 candidate-horizon comparisons with 0 mismatches, and the run is validated (`FULL_EXPERIMENT_VALID: true`). By the pre-registered Set-C rule (median Jaccard ≥ 0.8 and cross-continuation regret ≤ 1 miss for $\geq 80\%$ of LRU-optimal candidates), both MRU and mean-random continuation classify ROBUST at this sampled scope. MRU is very close to LRU: Set-C mean Jaccard is 0.9958 at capacity 32 and 0.9999 at capacity 128, with mean cross-continuation regret 0.0018 and 0.0001 misses, respectively. Mean-random continuation also classifies ROBUST, but it is visibly more perturbing: mean Jaccard is 0.7251 at capacity 32 and 0.8522 at capacity 128, with mean cross-continuation regret 0.0463 and 0.0197.

Full-population MRU census. The full-population MRU census (`analysis/continuation_policy_mru_population_census_20260914/validated/20260914T042528Z_1a29e773a113/`) extends the MRU side of that result to all five families, capacities 32, 64, 128, and 256, and horizons 4, 8, and 16. The census validates 60/60 chunks and 2,363,286/2,363,286 decision-horizon records, with 0 duplicate, missing, or extra decision-horizon keys; 30/30 mandatory gates and an independent recheck pass, and `FULL_CENSUS_VALID: true`. Across all decision-horizon units, mean optimal-set Jaccard is 0.9994054593, median Jaccard is 1.0, 98.69% of records have exact Jaccard 1, mean cross-continuation regret is 0.000304471 misses, median and p95 regret are both 0, the maximum mean regret is 0.193548 misses, and 99.9708% of LRU-optimal

candidates remain MRU-optimal on average. Strict preference reversals are rare: 4,646 discordant candidate pairs out of 24,791,643,360 pairwise comparisons, a fraction of 1.87×10^{-7} .

Informative-stratum result. Aggregate numbers alone would be misleading because both-tied decisions are common: 67.31% of all decision-horizon records are all-tied under both LRU and MRU. The pre-registered decisive stratum is therefore Set C, excluding decisions already all-tied under LRU. In this informative stratum, the census has 764,282 decision-horizon units from 299,905 eviction decisions. Set-C mean Jaccard is 0.9984587309, median Jaccard is 1.0, the strict reversal fraction is 5.38×10^{-7} , mean cross-continuation regret is 0.000638213 misses, and the mean fraction of LRU-optimal candidates still MRU-optimal is 0.9993955215. Since wiki2018 contributes no Set-C records, the Set-C result is not inflated by the degenerate negative control; the all-family and excluding-wiki Set-C summaries are identical.

Capacity and horizon effects. Sensitivity is largest at the smallest capacity and longest horizon, but remains small in absolute terms. By capacity in Set C, mean Jaccard and mean cross-continuation regret are 0.994809/0.002067 at capacity 32, 0.998859/0.000530 at capacity 64, 0.999806/0.000096 at capacity 128, and 0.999959/0.000019 at capacity 256. Thus the new population-only capacities, 64 and 256, reveal no hidden sensitivity and cross no pre-registered robustness threshold. By horizon in Set C, mean Jaccard declines modestly from 0.999820 at $H = 4$ to 0.999222 at $H = 8$ and 0.996869 at $H = 16$, while mean cross-continuation regret rises from 0.000039 to 0.000283 and 0.001355 misses. The direction is real but the magnitude remains far below a practically meaningful miss-count change.

Worst cell and scope. The worst Set-C family/capacity/horizon cell is cloudphysics/capacity 32/ $H = 16$: mean Jaccard 0.980433, median Jaccard 1.0, mean cross-continuation regret 0.003899 misses, and strict reversal fraction 3.45×10^{-5} . Even this cell satisfies the pre-registered robustness criterion. These results address the reviewer concern within the evaluated scope: sampled MRU and random at capacities 32/128, and full-population MRU at capacities 32/64/128/256. They do not establish population-scale random-continuation robustness, and they do not show that every possible deployed continuation policy would preserve the same ordering.

14. Practical Implications (RQ5)

This section synthesizes Sections 9–13 into guidance, without turning exploratory correlations into universal rules.

When should the offline counterfactual signal be trusted? Most directly in workload/capacity regimes with high capacity-scale reuse – identifiable, before running any closed-loop replay, from a low trace-derived all-tied fraction or a high trace-derived LRU hit rate (Section 12). In this dataset that means metacdn- and twemcache-like regimes, where offline discriminativeness is high and offline $\leftrightarrow$ closed-loop agreement is strongest.

When should a researcher be cautious? In near-zero-offline-signal regimes (cloudphysics/cap32 is the concrete example here): a small offline regret gap can flip sign without indicating a real disagreement – it is consistent with, not evidence against, offline informativeness elsewhere. Caution is also warranted around cold-start-scored windows (MetaKV) and validation-window-only evidence (MetaCDN), both stated explicitly in Section 4 rather than left implicit.

What does a high tie rate mean? Not that the target is useless – Section 9’s aggregate all-tied fraction of 0.68 coexists with genuinely discriminative sub-regimes. A high tie rate is itself informative: it identifies where a policy comparison is unlikely to be resolvable by this label family at all (wiki2018 is the limiting case), which is useful information for deciding where *not* to expect a learned method to outperform a heuristic on this evidence.

Why is closed-loop evaluation still necessary? Because the offline $\leftrightarrow$ closed-loop correspondence in Section 11 is real but bounded: it holds in most, not all, regimes, rests on $n = 8$ –10 independent cells, and has at least one exception (metakv/cap128) that is not explained by existing evidence. A result derived only from offline labels cannot be presented as a closed-loop performance claim.

How should continuation-policy assumptions be interpreted? As a tested, not assumed, robustness property, and one with real limits: the labels are close to unaffected by switching to MRU continuation at population scale across capacities 32/64/128/256, more (though still boundedly) affected by random continuation in the validated sample at capacities 32/128, and the sampled random effect grows somewhat with horizon (Section 13). This supports using the canonical LRU-continuation labels with reasonable confidence in the evaluated regimes, not a claim that continuation choice is irrelevant.

What capacity-scale reuse tells you before running anything.

The single trace statistic most predictive of both offline informativeness and closed-loop policy separation in this evaluation is a simple, cheaply computed one – trace-derived LRU hit rate at the target capacity (Section 12) – which a practitioner can compute on a new trace before committing to either an offline-label study or a full closed-loop evaluation of it.

None of the correlations underlying this guidance ($r = 0.6$ – 0.97 depending on which one) are treated here as anything stronger than the $n = 8$ – 10 exploratory findings they are; they inform where to look next, not a guarantee of what will be found there.

15. Discussion

What LAFC-Evict enables. A reusable, candidate-level, finite-horizon counterfactual supervision surface, materialized once from real traces and usable across value-regression, best-candidate-selection, and pairwise formulations (Sections 2–8) without every study re-deriving its own labels from raw traces (Section 17 situates this against prior learned-cache work, most of which derives labels internally and non-reusably).

What the evaluation reveals, beyond the surface itself. Four findings that only emerge from evaluating the surface against closed-loop behavior, not from describing the surface alone: the target’s discriminativeness is workload-structured, not incidental (Section 9); that structure is explained, not just observed, by a single trace-level property, capacity-scale reuse (Section 12); offline and closed-loop policy orderings agree in most, not all, informative regimes, with the exceptions identified rather than smoothed over (Section 11); and the canonical LRU-continuation assumption is robust, with quantified limits, at the capacities tested (Section 13).

Why the negative/degenerate regimes are scientifically useful, not an embarrassment. wiki2018’s complete degeneracy and cloudphysics/cap32’s near-zero offline signal are both, in this paper, explained results (Section 12), not unexplained anomalies. A benchmark whose degenerate cases are mechanistically understood is more trustworthy in its non-degenerate claims than one that reports only favorable cases; this is the paper’s answer to the historical criticism that the target’s tie-heaviness was previously stated but not investigated.

Offline surrogate evaluation vs. closed-loop performance. This paper’s position, supported by Sections 11 and 10, is that the offline surrogate

is a genuine, partially-validated predictor of closed-loop policy separation in identifiable regimes, and that this relationship is itself the contribution – not a claim that the surrogate can replace closed-loop evaluation. The two exceptions found (cloudphysics/cap32, metakv/cap128) are not edge cases invented for balance; they were identified by the same symmetric procedure that found the 8 agreeing cells, and metakv/cap128 in particular remains only partially understood even after dedicated mechanistic analysis (Section 12).

Continuation robustness as currently validated. The robustness finding in Section 13 is deliberately reported with its exact scope attached: sampled MRU and mean-random continuation at capacities 32/128, and full-population MRU continuation at capacities 32/64/128/256. Within that scope, the LRU continuation used for label generation is not driving the target ordering to a practically meaningful degree: Set-C median Jaccard remains 1.0, mean cross-continuation regret is below 0.001 misses in the population MRU census, and capacities 64/256 do not reveal hidden sensitivity. The result does not say that every stateful deployed continuation policy would preserve the same ordering; it says the tested alternatives did, with random tested only in the validated sample.

Methodological limitations that shape every result above are collected in Section 16 rather than repeated here; Section 9’s target-discriminativeness numbers, Section 11’s small n , and Section 13’s continuation-policy scope are the three limitations most load-bearing for the claims in this discussion specifically.

16. Limitations, Ethics, and Reuse

This evaluation surface has several important constraints, and the practical guidance in Section 14 should be read together with them rather than instead of them.

First, the main label is a finite-horizon, continuation-policy-specific counterfactual miss cost, not an offline-optimal label for the full future control problem. Good performance on this label does not automatically imply on-line deployment gains, which is exactly why Sections 10 and 11 evaluate closed-loop behavior directly rather than relying on the offline label alone.

Second, the canonical labels use LRU as the continuation policy after a forced eviction. Section 13 reports two bounded continuation-policy checks: a sampled study at capacities 32 and 128, where both MRU and mean-random continuation classify as robust on the pre-registered rule, and a full-

population MRU census across capacities 32, 64, 128, and 256, which also satisfies the MRU robustness criterion. This does not prove continuation-independence in general. Random continuation is not validated at population scale, and no experiment here covers every possible stateful deployment policy.

Third, the target is substantially tied. Across the full audited canonical dataset, the all-tied fraction is approximately 0.68 and the unique-winner fraction is exactly 0.0 – no audited decision in this dataset has a single uniquely-best candidate victim. Discriminativeness varies strongly by family, capacity, and horizon; one family, wiki2018, is a fully degenerate negative control at every capacity and horizon tested, with zero reuse events in its scored windows, which is a mathematical certainty rather than a statistical property of that trace.

Fourth, the evaluated dataset uses unit-object, count-based capacities (32, 64, 128, or 256 objects) with unconditional admission of the requested object. Variable object sizes, byte-weighted miss cost, latency-aware cost, and dirty-write cost are not modeled by this label family, and admission is never withheld. Claims in this paper should be read as scoped to this abstraction and not generalized to byte-addressable or admission-gated cache models without further evidence.

Fifth, closed-loop evidence itself has two family-specific caveats that must be carried alongside any result that uses it. MetaCDN’s closed-loop evidence is validation-window evidence, not a temporally held-out test window (Twemcache’s is test-window). MetaKV is the only family whose scored window begins at an empty cache (a cold start, identifiable from the identity evictions = misses - capacity); this is plausibly, but not conclusively, connected to MetaKV being the one Tier-1 cell where LRU falls to third place, behind both SIEVE and random-mean (Section 12).

Sixth, a known data-generation issue remains open: in the audited canonical data, `candidate_is_predictor_victim` is identical to `candidate_is_lru_victim` throughout, and related predictor/ bucket/confidence fields include constant or placeholder values. This paper does not make any claim that depends on those predictor fields being informative.

Seventh, benchmark reuse must preserve provenance distinctions. The release exposes generated supervision and derived views, but it does not erase the attribution, citation, or licensing requirements of the upstream raw traces from which candidate rows were derived; those raw traces remain external artifacts with their own governance constraints. The open subset is

also intentionally narrower than the full generated corpus for redistribution reasons, and not every selected family appears in every split of the evaluated release package.

Finally, this paper reports closed-loop results for LRU, MRU, random, and SIEVE (Section 10); it does not report a closed-loop evaluation of any learned policy trained on these labels. A frozen pilot (not the production Tier-1 evaluation reported here) previously found a simple learned policy competitive with random and SIEVE but not with LRU on one family, with the other pilot family blocked on the absence of an independent test window; neither result is repeated or extended here, and learned-policy closed-loop competitiveness remains future work rather than a claim of this paper.

17. Related Work

17.1. Production Cache Workloads and Benchmark Traces

Public traces and benchmark infrastructure provide the substrate for cache evaluation. The UMass Trace Repository and large-scale workload studies of key-value stores and Twemcache clusters demonstrate the value of realistic, heterogeneous request streams for systems research [14, 15, 13]. CacheLib further shows what that heterogeneity looks like from the inside of a single production caching engine, which must serve many such workloads simultaneously behind one shared implementation [16]. More broadly, benchmark artifacts such as YCSB, OLTP-Bench, and the Join Order Benchmark made comparison easier by publishing reusable workloads, interfaces, and evaluation surfaces rather than one-off experiments [17, 18, 19]. Cache-specific infrastructure such as libCacheSim and `cache_dataset` further lowers the cost of trace-driven evaluation by providing high-performance simulation support and large open trace collections [20, 21]. Existing trace libraries and simulators therefore supply essential raw material for cache studies, and some public artifacts even expose oracle-style features or procedural oracle policies. LAFC-Evict packages a different layer of supervision: for each full-cache miss decision and candidate victim, it releases finite-horizon counterfactual loss labels under a fixed continuation policy, together with decision-level and pairwise views.

Those distinctions matter because raw traces, workload studies, and simulator-oriented artifacts do not by themselves provide a reusable table of candidate-level finite-horizon counterfactual outcomes. LAFC-Evict is intended to complement that ecosystem rather than replace it. The artifact is

useful precisely when a downstream user wants to study offline value regression, best-candidate selection, preference learning, or label-distribution analysis without first regenerating the same counterfactual rollouts from scratch.

17.2. Learned Cache Replacement

Recent learned-caching papers contribute online policies, policy-selection schemes, or learning frameworks rather than reusable supervised benchmarks, and they differ in what signal they learn from and at what granularity. LRB learns a relaxed Belady-style policy for CDN caching, and LHD instead models hit density directly for key-value caches [7, 22]. GL-Cache and 3L-Cache both target the efficiency and overhead profile of learned eviction, at group and object granularity respectively, so that a learned policy stays cheap enough to run inline [9, 11]. LeCaR learns between replacement experts online [6], and CACHEUS extends that practical line with adaptive expert selection, giving both a mechanism for adjusting to changing workload regimes without full retraining [10]. Parrot uses imitation learning to approximate Belady-style replacement decisions from past accesses [8], while Hawkeye and Mockingjay show how Belady-derived signals can guide deployable replacement policies in hardware caches [23, 24]. MAT focuses on reducing the overhead of ML-guided eviction by using a heuristic filter around a learned module [25]. ALPS emphasizes lightweight learned replacement and the practical cost of learning overheads [26], while Feedforward Neural Networks for Caching is useful precisely because it questions whether heavier learned models are always justified [27]. HR-Cache extends the learned-caching line into edge delivery settings [28].

HALP is a closer conceptual precedent worth singling out: it deploys a preference-learning eviction policy in production for YouTube’s CDN, where a lightweight reward model is trained continuously from automated preference comparisons between candidate evictions – a scheme the authors liken to reinforcement learning from feedback rather than supervised imitation of a fixed oracle [12]. This makes HALP the closest production system to LAFC-Evict’s own pairwise derived view, but the two are not the same kind of artifact and should not be read as such. HALP’s preference signal is generated and consumed online, as the primary training objective of a deployed policy; LAFC-Evict’s pairwise view is instead a static, deterministic reformatting of candidate-level finite-horizon counterfactual labels that already exist independently of it (Section 2), released so pairwise/ranking-style methods can consume the same underlying supervision. LAFC-Evict does not claim any

preference-learning contribution of its own – HALP already demonstrates that preference signals are useful for eviction in production; LAFC-Evict’s contribution is a reusable, decoupled label surface, not a new instance of preference learning. Baleen addresses a related but distinct problem, applying coordinated ML admission and prefetching – rather than eviction – to flash caches, and end-to-end optimizes for backend load rather than a per-candidate label [29]. It is included here as a reminder that objective design (what to admit and prefetch, not only what to evict) is a live and separate axis from the counterfactual-label problem LAFC-Evict addresses, not as a directly comparable eviction system.

Across all of these works, the primary contribution is a policy, a policy-selection scheme, or an online learning framework. By contrast, LAFC-Evict releases a static benchmark surface with candidate-level finite-horizon counterfactual labels, decision-view summaries, and a shipped pairwise sample for offline value regression, best-candidate selection, and preference-learning tasks. To our knowledge, we are not aware of a public artifact in this line that releases reusable per-candidate finite-horizon counterfactual supervision at this scale.

17.3. Modern Heuristic and Non-Learning Eviction

Classical cache-replacement work defines the policy landscape that any learned benchmark must eventually interface with. Belady’s look-ahead policy remains the canonical offline reference point [30]. Offline analyses such as FOO/PFOO compute practical bounds on optimal caching with variable object sizes, but their artifacts target trace-level optimality analysis rather than a reusable per-decision, per-candidate supervised label surface [31]. Practical online policies then emphasize different signals: LIRS focuses on inter-reference recency [2], ARC adaptively balances recency and frequency [1], CLOCK-Pro revisits CLOCK-style replacement with stronger reuse sensitivity [32], and GreedyDual-Size accounts for cost and size [33]. TinyLFU and W-TinyLFU further show that admission and segmented cache management matter in modern software caches [34, 35]. More recent object-cache work shows that policy progress does not always come from heavier machinery: S3-FIFO argues that simple FIFO-based queueing can beat more elaborate baselines at scale [4], and a companion analysis explains *why* at a mechanism level – lazy promotion and quick demotion, not queue discipline by itself, are what let FIFO-derived policies beat LRU-style recency tracking [36]. SIEVE emphasizes turn-key deployability for web caches [3] and,

unlike S3-FIFO, is not only discussed here: it is one of the four closed-loop policy arms (alongside LRU, MRU, and random-mean) evaluated directly in Section 10. LAH revisits this same simple-heuristic family from the other direction: rather than proposing a new heuristic, it learns the *parameters* of an existing one – S3-FIFO, instantiated as S4-FIFO – from a control plane decoupled from the data plane, reporting robustness and interpretability gains over both purely heuristic and heavier learned baselines [5]. S3-FIFO, the lazy-promotion/quick-demotion analysis, and LAH are strong online comparison targets for future end-to-end studies, but only SIEVE among this group is evaluated here; LAFC-Evict does not claim a new eviction policy of its own. The contribution here is reusable supervision that can be used to train or diagnose such policies, not an online replacement rule that supersedes them.

17.4. *Learning-Augmented Caching Theory*

Another relevant line of work treats caching as an online problem augmented by predictions rather than as a supervised dataset, with provable robustness/consistency guarantees as the object of study rather than empirical miss ratio alone – a genuinely different research question from the learned-from-data policies of the previous two subsections, and one LAFC-Evict is careful not to conflate with them. Competitive Caching with Machine Learned Advice and later learning-augmented caching results formalize how predictive advice can improve classical online algorithms while preserving worst-case robustness guarantees [37, 38]. Robust Learning-Augmented Caching: An Experimental Study extends that line empirically, testing how the theoretical robustness/consistency trade-off behaves under realistic, imperfect predictors rather than only in the adversarial worst case [39], while Parsimonious Learning-Augmented Caching studies how much prediction access is really needed to retain the theoretical benefit [40]. LAH (previous subsection) sits at a practical bridge between this theoretical line and the learned-from-data policies above: it carries no formal competitive-ratio guarantee, but its explicit robustness and interpretability goals echo the same underlying concern that a learned or advice-driven component should degrade gracefully, not fail silently, when its signal is wrong [5]. This perspective is complementary to LAFC-Evict rather than directly competing with it. Learning-augmented caching papers typically study online algorithms equipped with advice, predictor queries, and robustness objectives. By contrast, LAFC-Evict releases finite-horizon, continuation-policy-dependent

counterfactual labels for every candidate within a decision. The artifact can train or diagnose predictors and eviction-scoring models before they are embedded into an online algorithm, and it supports value regression, best-candidate selection, pairwise preference learning, and label characterization from one release.

17.5. Offline Surrogate Evaluation versus Deployed, Closed-Loop Behavior

A methodological question this paper shares with several other applied-ML domains – recommendation, autonomous driving, and reinforcement learning among them – is whether an offline proxy metric, computed without deploying a policy, predicts that policy’s actual closed-loop performance. Within caching specifically, Qiu, Yang, and Harchol-Balter give a sharp, caching-native instance of this same gap: increasing a cache’s hit ratio can, past a certain point, *decrease* end-to-end throughput and increase response time, because a higher hit ratio changes contention on the hit path itself [41]. That result targets a different offline/online pairing than this paper does – cache hit ratio versus downstream system throughput, rather than finite-horizon counterfactual miss cost versus closed-loop miss ratio – and this paper does not claim it studies the same quantity. What it establishes, directly and independently, is that a favorable cache-local metric need not imply favorable end-to-end system behavior – exactly the caution that motivates treating Sections 10 and 11’s closed-loop validation as necessary, rather than inferring closed-loop behavior from the offline labels alone. The broader cross-domain finding, outside caching, is the same in spirit: offline and online/closed-loop metrics correlate imperfectly and domain-specifically across recommendation, autonomous driving, and reinforcement learning, which is why closed-loop or online validation is treated as necessary rather than optional wherever it is feasible. This paper’s Sections 11 and 10 pose the same question for finite-horizon counterfactual cache-eviction labels specifically, with a bounded, regime-characterized answer rather than a universal one; beyond the hit-ratio-versus-throughput result above, to the best of our knowledge we are not aware of prior work posing this offline-versus-closed-loop question for candidate-level cache-eviction supervision in particular. A broader cross-domain literature search for this methodological category outside caching (surrogate/offline-vs-closed-loop evaluation methodology as a topic in its own right, in recommendation/autonomous-driving/RL) remains only partially complete at the time of writing; see `docs/PE_LITERATURE_SEARCH_GAPS.md` in the accompanying artifact for what was and was not verified.

17.6. Positioning LAFC-Evict

Public learning-to-rank collections such as LETOR and the Yahoo! Learning to Rank Challenge show how reusable labeled artifacts can organize a research community [42, 43, 44]. In that limited sense, LAFC-Evict is to learned cache eviction somewhat analogous to LETOR-style ranking corpora: a standardized candidate-level label surface reduces bespoke label generation and evaluation drift. The label design in LAFC-Evict also connects to counterfactual learning and ranking because each cache decision induces a candidate set, a minimum-loss subset, regret summaries, and a natural pairwise structure [45, 46, 47, 48]. Even so, LAFC-Evict is not simply a ranking dataset transplanted into systems. Its labels are tied to finite-horizon eviction outcomes under a specified continuation policy.

To state the boundary precisely, given the full literature surveyed above: LAFC-Evict does not claim to be the first to use future accesses for supervision (LRB, Parrot, and HALP all do so already); the first to support pairwise or preference-style modeling of cache decisions (HALP, again, does this directly and online); the first learned or learning-augmented eviction approach (the entire learned-cache-replacement and learning-augmented-theory literature surveyed above); or the first to raise offline-versus-closed-loop validity as a methodological concern (established in other applied-ML domains, and now also directly in caching via the hit-ratio/throughput result). What is not, to our knowledge, already available is a public cache-eviction artifact that releases reusable candidate-level finite-horizon counterfactual supervision together with decision-level and pairwise task views, at production scale, decoupled from any specific downstream learned method. In that narrower sense, LAFC-Evict is neither a new online policy nor merely a raw trace repository or simulator-centric contribution, but a reusable benchmark artifact for learned cache-eviction research that complements existing traces, simulators, learned and heuristic policy papers, and learning-augmented theory alike.

18. Conclusion

This paper positions LAFC-Evict as a benchmark artifact for learned cache eviction rather than as a new policy paper. The core contribution is a release-oriented package built around candidate-level counterfactual supervision, derived decision and pairwise tasks, and validation-aware artifact construction. The preserved open release now couples manifest-backed scale

and successful end-to-end validation with committed summaries of candidate-label distributions, a lightweight linear value-regression baseline, a derived best-candidate selector, and pairwise-sample-backed feature baselines that show measurable decision-relevant signal in the released candidate features.

Those results warrant conservative interpretation. The linear regression baseline is weak, the best-candidate selector is a reproducible benchmark floor rather than deployment evidence, the pairwise results remain backed by the shipped capped sample rather than any full quadratic pairwise universe, and the labels remain finite-horizon counterfactuals rather than globally optimal online targets. The validated continuation studies substantially reduce concern that the canonical LRU continuation alone drives the reported target ordering, but only within the tested scope: sampled MRU/random at capacities 32/128 and population MRU at capacities 32/64/128/256. Future extensions can focus on stronger learned baselines, online policy studies that sit above the benchmark layer, broader continuation policies, and broader public hosting where redistribution constraints permit it.

Acknowledgments

The author gratefully acknowledges Prof. Ioannis (Yiannis) Koutis for his guidance and support. The author also acknowledges support from the Google Cloud Research Credits Program; Cohere Labs through the Catalyst Grant Program, which provided \$1,000 in API credits; and CloudRift AI through the AI Builder Grant, which provided 1,000 credits. The author further thanks Amazon Web Services, through the AWS Open Data Sponsorship Program, for sponsored hosting of up to 600 GB for two years to support public access to LAFC-Evict.

Declaration of competing interest

Data availability

The evaluated LAFC-Evict open-release snapshot (candidate rows, decision view, pairwise sample, manifest, and checksums; Section 7) is developed in the project’s public GitHub repository at <https://github.com/SoroushVahidi/lafc-evict-dataset> and archived under Zenodo concept DOI 10.5281/zenodo.21895843, released under a CC0-1.0 license, per the repository’s CITATION.cff.

Funding

This work received computing/API support through the Google Cloud Research Credits Program, the Cohere Labs Catalyst Grant Program (\$1,000 in API credits), and the CloudRift AI Builder Grant (1,000 credits). Public hosting of the LAFC-Evict dataset is supported by the AWS Open Data Sponsorship Program, which provides sponsored hosting of up to 600 GB for two years.

CRedit authorship contribution statement

Soroush Vahidi: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Declaration of Generative AI and AI-assisted technologies in the writing process

The authors used generative AI and AI-assisted coding and search tools to support manuscript drafting and editing, related-work exploration, code development, debugging, experiment organization, and result and table summarization. All scientific claims, related-work interpretations, code, experiments, tables, and final manuscript text were reviewed and verified by the authors, who remain fully responsible for the content and correctness of this work. No AI tool is listed as an author or as a contributor to authorship.

Appendix A. Additional Release Tables

This appendix carries release/schema-descriptive tables that support reproducibility but do not themselves answer a research question posed in the main text (per the figure/table planning discipline in `docs/PE_FIGURE_TABLE_PLAN.md` in the accompanying artifact: avoid main-text tables that read as dataset-schema reporting rather than scientific results). Content is unchanged from the underlying release manifest; nothing here was recomputed for this appendix.

References

- [1] N. Megiddo, D. S. Modha, Arc: A self-tuning, low overhead replacement cache, in: Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST), 2003, pp. 115–130.
URL <https://www.usenix.org/conference/fast-03/arc-self-tuning-low-overhead-replacement-cache>
- [2] S. Jiang, X. Zhang, Lirs: An efficient low inter-reference recency set replacement policy to improve buffer cache performance, in: Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, 2002, pp. 31–42. doi:10.1145/511334.511340.
- [3] Y. Zhang, J. Yang, Y. Yue, Y. Vigfusson, K. V. Rashmi, SIEVE is simpler than LRU: an efficient Turn-Key eviction algorithm for web caches, in: 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24), USENIX Association, 2024, pp. 1229–1246.
URL <https://www.usenix.org/conference/nsdi24/presentation/zhang-yazhuo>
- [4] J. Yang, Y. Zhang, Z. Qiu, Y. Yue, R. Vinayak, Fifo queues are all you need for cache eviction, in: ACM SIGOPS 29th Symposium on Operating Systems Principles, SOSP '23, 2023. doi:10.1145/3600006.3613147.
URL https://www.pdl.cmu.edu/PDL-FTP/Storage/FIFOqueues-SOSP23_abs.shtml
- [5] H. Xia, W. Nixon, B. D. Marthen, P. Bhandari, J. Yang, Learning-augmented heuristics: Simple, yet smart, robust and interpretable cache eviction, in: 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26), USENIX Association, 2026.
URL <https://www.usenix.org/conference/osdi26/presentation/xia>
- [6] G. Vietri, L. V. Rodriguez, W. A. Martinez, S. Lyons, J. Liu, R. Rangaswami, G. Narasimhan, M. Zhao, Driving cache replacement with ml-based lecar, in: Proceedings of the USENIX Workshop on Hot Topics in Storage Systems (HotStorage), 2018.

URL <https://www.usenix.org/conference/hotstorage18/presentation/vietri>

- [7] Z. Song, D. S. Berger, K. Li, W. Lloyd, Learning relaxed belady for content distribution network caching, in: 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20), USENIX Association, Santa Clara, CA, 2020, pp. 529–544.
URL <https://www.usenix.org/conference/nsdi20/presentation/song>
- [8] E. Liu, M. Hashemi, K. Swersky, P. Ranganathan, J. Ahn, An imitation learning approach for cache replacement, in: Proceedings of the 37th International Conference on Machine Learning, Vol. 119 of Proceedings of Machine Learning Research, PMLR, 2020, pp. 6237–6247.
URL <https://proceedings.mlr.press/v119/liu20f.html>
- [9] J. Yang, Z. Mao, Y. Yue, K. V. Rashmi, GL-Cache: Group-level learning for efficient and high-performance caching, in: 21st USENIX Conference on File and Storage Technologies (FAST 23), USENIX Association, Santa Clara, CA, 2023, pp. 115–134.
URL <https://www.usenix.org/conference/fast23/presentation/yang-juncheng>
- [10] L. V. Rodriguez, F. Yusuf, S. Lyons, E. Paz, R. Rangaswami, J. Liu, M. Zhao, G. Narasimhan, Learning cache replacement with cacheus, in: 19th USENIX Conference on File and Storage Technologies (FAST 21), USENIX Association, 2021, pp. 341–354.
URL <https://www.usenix.org/conference/fast21/presentation/rodriguez>
- [11] W. Zhou, Z. Niu, Y. Xiong, J. Fang, Q. Wang, 3L-Cache: Low overhead and precise learning-based eviction policy for caches, in: 23rd USENIX Conference on File and Storage Technologies (FAST 25), USENIX Association, Santa Clara, CA, 2025, pp. 237–254.
URL <https://www.usenix.org/conference/fast25/presentation/zhou-wenbin>
- [12] Z. Song, K. Chen, N. Sarda, D. Altınbüken, E. Brevdo, J. Coleman, X. Ju, P. Jurczyk, R. Schooler, R. Gummadi, HALP: Heuristic aided learned preference eviction policy for YouTube content delivery network,

- in: 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23), USENIX Association, 2023, pp. 1149–1163.
URL <https://www.usenix.org/conference/nsdi23/presentation/song-zhenyu>
- [13] J. Yang, Y. Yue, K. V. Rashmi, A large-scale analysis of hundreds of in-memory cache clusters at twitter, in: 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), 2020, pp. 191–208.
URL <https://www.usenix.org/conference/osdi20/presentation/yang>
- [14] University of Massachusetts Amherst, Umass trace repository, official trace repository website (2025).
URL <https://traces.cs.umass.edu/>
- [15] B. Atikoglu, Y. Xu, E. Frachtenberg, S. Jiang, M. Paleczny, Workload analysis of a large-scale key-value store, in: Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, 2012, pp. 53–64.
- [16] B. Berg, D. S. Berger, S. McAllister, I. Grosof, S. Gunasekar, J. Lu, M. Uhlar, J. Carrig, N. Beckmann, M. Harchol-Balter, G. R. Ganger, The CacheLib caching engine: Design and experiences at scale, in: 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), USENIX Association, 2020, pp. 753–768.
URL <https://www.usenix.org/conference/osdi20/presentation/berg>
- [17] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, R. Sears, Benchmarking cloud serving systems with ycsb, in: Proceedings of the ACM Symposium on Cloud Computing, 2010, pp. 143–154.
- [18] D. E. Difallah, A. Pavlo, C. Curino, P. Cudré-Mauroux, Oltp-bench: An extensible testbed for benchmarking relational databases, Proceedings of the VLDB Endowment 7 (4) (2013) 277–288.
- [19] V. Leis, B. Radke, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, T. Neumann, Query optimization through the looking glass, and what

- we found running the join order benchmark, *The VLDB Journal* 27 (5) (2018) 643–668. doi:10.1007/s00778-017-0480-7.
- [20] cacheMon, libCacheSim: A high performance cache simulator and library, <https://github.com/cacheMon/libCacheSim>, accessed 2026-07-03 (2026).
 - [21] cacheMon, A comprehensive open-source cache trace dataset, https://github.com/cacheMon/cache_dataset, accessed 2026-07-03 (2026).
 - [22] N. Beckmann, H. Chen, A. Cidon, LHD: Improving cache hit rate by maximizing hit density, in: 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18), USENIX Association, Renton, WA, 2018, pp. 389–403.
URL <https://www.usenix.org/conference/nsdi18/presentation/beckmann>
 - [23] A. Jain, C. Lin, Back to the future: Leveraging belady’s algorithm for improved cache replacement, in: Proceedings of the 43rd International Symposium on Computer Architecture, 2016, pp. 78–89. doi:10.1145/3007787.3001146.
 - [24] I. Shah, A. Jain, C. Lin, Effective mimicry of belady’s min policy, in: 2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2022, pp. 558–572. doi:10.1109/HPCA53966.2022.00048.
 - [25] D. Yang, D. S. Berger, K. Li, W. Lloyd, A learned cache eviction framework with minimal overhead (2023). arXiv:2301.11886, doi:10.48550/arXiv.2301.11886.
URL <https://arxiv.org/abs/2301.11886>
 - [26] R. Shahout, R. Friedman, Sketching the path to efficiency: Lightweight learned cache replacement, in: 27th International Conference on Principles of Distributed Systems (OPODIS 2023), Vol. 286 of Leibniz International Proceedings in Informatics (LIPIcs), 2024, pp. 34:1–34:21. doi:10.4230/LIPIcs.OPODIS.2023.34.
 - [27] V. Fedchenko, G. Neglia, B. Ribeiro, Feedforward neural networks for caching: Enough or too much?, *SIGMETRICS Performance Evaluation Review* 46 (3) (2018) 139–142. doi:10.1145/3308897.3308958.

- [28] H. Torabi, H. Khazaei, M. Litoiu, A learning-based caching mechanism for edge content delivery, in: Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering, 2024, pp. 236–246. doi:10.1145/3629526.3645037.
- [29] D. L.-K. Wong, H. Wu, C. Molder, S. Gunasekar, J. Lu, S. Khandkar, A. Sharma, D. S. Berger, N. Beckmann, G. R. Ganger, Baleen: ML admission & prefetching for flash caches, in: 22nd USENIX Conference on File and Storage Technologies (FAST 24), USENIX Association, 2024, pp. 347–371.
URL <https://www.usenix.org/conference/fast24/presentation/wong>
- [30] L. A. Belady, A study of replacement algorithms for a virtual-storage computer, IBM Systems Journal 5 (2) (1966) 78–101. doi:10.1147/sj.52.0078.
- [31] D. S. Berger, N. Beckmann, M. Harchol-Balter, Practical bounds on optimal caching with variable object sizes, Proceedings of the ACM on Measurement and Analysis of Computing Systems 2 (2) (2018). doi:10.1145/3224427.
URL <https://doi.org/10.1145/3224427>
- [32] S. Jiang, F. Chen, X. Zhang, Clock-pro: An effective improvement of the clock replacement, in: Proceedings of the Annual USENIX Technical Conference, 2005, pp. 323–336.
URL <https://www.usenix.org/conference/2005-usenix-annual-technical-conference/clock-pro-effective-improvement-clock-replacement>
- [33] P. Cao, S. Irani, Cost-aware www proxy caching algorithms, in: Proceedings of the USENIX Symposium on Internet Technologies and Systems, 1997, pp. 193–206.
URL <https://www.usenix.org/conference/usits-97/cost-aware-www-proxy-caching-algorithms>
- [34] G. Einziger, R. Friedman, B. Manes, TinyLfu: A highly efficient cache admission policy, ACM Transactions on Storage 13 (4) (2017) 35:1–35:31. doi:10.1145/3149371.

- [35] G. Einziger, O. Eytan, R. Friedman, B. Manes, Adaptive software cache management, in: Proceedings of the 19th International Middleware Conference, 2018, pp. 94–106. doi:10.1145/3274808.3274816.
- [36] J. Yang, Z. Qiu, Y. Zhang, Y. Yue, K. V. Rashmi, FIFO can be better than LRU: The power of lazy promotion and quick demotion, in: 19th Workshop on Hot Topics in Operating Systems (HotOS 23), ACM, 2023. doi:10.1145/3593856.3595887.
- [37] T. Lykouris, S. Vassilvitskii, Competitive caching with machine learned advice, *Journal of the ACM* 68 (4) (2021) 24:1–24:25. doi:10.1145/3447579.
- [38] A. Wei, Better and simpler learning-augmented online caching, in: Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2020), Vol. 176 of Leibniz International Proceedings in Informatics (LIPIcs), Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, pp. 60:1–60:17. doi:10.4230/LIPIcs.APPROX/RANDOM.2020.60.
URL <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.APPROX/RANDOM.2020.60>
- [39] J. Chłedowski, A. Polak, B. Szabucki, K. T. Żoła, Robust learning-augmented caching: An experimental study, in: Proceedings of the 38th International Conference on Machine Learning, Vol. 139 of Proceedings of Machine Learning Research, PMLR, 2021, pp. 1920–1930.
URL <https://proceedings.mlr.press/v139/chledowski21a.html>
- [40] S. Im, R. Kumar, A. Petety, M. Purohit, Parsimonious learning-augmented caching, in: Proceedings of the 39th International Conference on Machine Learning, Vol. 162 of Proceedings of Machine Learning Research, PMLR, 2022, pp. 9588–9601.
URL <https://proceedings.mlr.press/v162/im22a.html>
- [41] Z. Qiu, J. Yang, M. Harchol-Balter, Why increasing the hit ratio can hurt cache throughput, *Queueing Systems* 110 (2) (2026). doi:10.1007/s11134-026-09986-1.
- [42] T.-Y. Liu, Learning to rank for information retrieval, *Foundations and Trends in Information Retrieval* 3 (3) (2009) 225–331. doi:10.1561/15000000016.

- [43] T. Qin, T.-Y. Liu, J. Xu, H. Li, Letor: A benchmark collection for research on learning to rank for information retrieval, *Information Retrieval* 13 (4) (2010) 346–374. doi:10.1007/s10791-009-9123-y.
- [44] O. Chapelle, Y. Chang, T.-Y. Liu (Eds.), *Proceedings of the Yahoo! Learning to Rank Challenge*, Vol. 14 of JMLR Workshop and Conference Proceedings, JMLR.org, 2011.
URL <https://proceedings.mlr.press/v14/>
- [45] A. Swaminathan, T. Joachims, Counterfactual risk minimization: Learning from logged bandit feedback, in: *Proceedings of the 32nd International Conference on Machine Learning*, 2015, pp. 814–823.
URL <https://proceedings.mlr.press/v37/swaminathan15.html>
- [46] L. Li, W. Chu, J. Langford, X. Wang, Unbiased offline evaluation of contextual-bandit-based news article recommendation algorithms, in: *Proceedings of the 4th ACM International Conference on Web Search and Data Mining*, 2011, pp. 297–306. doi:10.1145/1935826.1935878.
- [47] T. Joachims, A. Swaminathan, T. Schnabel, Unbiased learning-to-rank with biased feedback, in: *Proceedings of the 10th ACM International Conference on Web Search and Data Mining*, 2017, pp. 781–789.
- [48] Y. Saito, Unbiased pairwise learning from biased implicit feedback, in: *Proceedings of the 2020 ACM SIGIR International Conference on Theory of Information Retrieval*, 2020, pp. 5–12. doi:10.1145/3409256.3409812.

Table 9: Offline $\leftrightarrow$ closed-loop correspondence and continuation-policy robustness, summarized. The continuation rows use Set C (discriminative-under-LRU). Sampled random evidence is available only at capacities 32/128; full-population evidence is MRU-only.

<i>Offline$\leftrightarrow$closed-loop, $H=16$ ($n=10$ cells; SIEVE excluded)</i>		
metric	MRU-vs-LRU	random-vs-LRU
Concordant / discordant / tied (of 10)	7/1/2	6/2/2
Pearson r	0.835	0.755
Spearman ρ	0.915	0.829
Kendall τ -b	0.773	0.591
<i>Sampled continuation robustness, Set C, by capacity</i>		
metric	cap. 32	cap. 128
MRU median / mean Jaccard	1.000 / 0.9958	1.000 / 0.9999
MRU mean cross-cont. regret	0.0018	0.0001
MRU strict reversal fraction	2.7×10^{-5}	0.0
random median / mean Jaccard	1.000 / 0.7251	1.000 / 0.8522
random mean cross-cont. regret	0.0463	0.0197
random strict reversal fraction	1.0×10^{-3}	1.5×10^{-6}
<i>Full-population MRU census, Set C</i>		
metric	all capacities	new capacities 64/256
Scope	764,282 decision-H units	391,499 decision-H units
median / mean Jaccard	1.000 / 0.998459	1.000 / 0.999425
mean cross-cont. regret	0.000638	0.000267
strict reversal fraction	5.38×10^{-7}	2.83×10^{-7}
LRU-opt. still MRU-opt. fraction	0.999396	0.999745
robustness classification	ROBUST	ROBUST

Table A.10: Decision-view-backed benchmark composition by family, capacity, horizon, and split.

dimension	counts
trace_family	wiki2018=598,560; cloudphysics=576,219; metakv=454,053; twemcache=397,494; metacdn=336,960
capacity	32=613,314; 64=599,172; 128=582,678; 256=568,122
horizon	4=787,762; 8=787,762; 16=787,762
split	test=242,418; train=1,746,882; val=373,986

Table A.11: Decision-view-backed candidate-count statistics. These values summarize the number of candidates per decision in the preserved release.

group	min	med	mean	max
overall	32	64	117.63	256
cloudphysics	32	64	118.35	256
metacdn	32	64	117.12	256
metakv	32	64	119.52	256
twemcache	32	64	111.52	256
wiki2018	32	64	119.85	256

Table A.12: Candidate-label and best-candidate-selection summaries from committed result JSON files.

statistic	value
candidate rows summarized	277,995,072
y_loss mean / std	7.7561 / 4.6542
y_loss q05 / q25 / q50 / q75 / q95	2 / 4 / 7 / 11 / 16
y_value mean / std	-7.7561 / 4.6542
y_value q05 / q25 / q50 / q75 / q95	-16 / -11 / -7 / -4 / -2
manifest-backed decision rows	2,363,286
best-candidate evaluator decision count	2,363,286 (exact match)
best-candidate optimal-selection rate	0.8753
best-candidate regret mean / median / max	0.1251 / 0 / 3
random-selector expected optimal-selection rate	0.9912
random-selector expected regret	0.0088